

Ecco il riassunto del tuo primo file sulle User Stories. L'ho rielaborato e "tradotto" in un linguaggio molto più diretto e masticabile, perfetto per studiare senza impazzire dietro le definizioni troppo formali!

1. La mentalità Agile: perché le User Stories?

- Il punto di partenza dell'Agile è che all'inizio del progetto né il cliente né gli sviluppatori sanno tutto: si impara strada facendo.
- Nel metodo tradizionale i requisiti si bloccano subito ed è un guaio (e un costo) cambiarli dopo, mentre i metodi Agili nascono proprio per accogliere i cambiamenti in qualsiasi momento.
- I requisiti non sono documenti infiniti e formali, ma frasi brevi e ad alto livello che indicano le cose più importanti (chiamate Features o User Stories).
- Per capire cosa vuole davvero il cliente, serve tantissima interazione. L'ideale è avere il cliente sempre a disposizione, o in alternativa un suo "sostituto" detto Product Owner (PO), che guida il progetto, collabora col team e bada agli obiettivi di business.
- Il software viene rilasciato spesso (si lavora in iterazioni di 1-2 settimane e si rilascia un programma funzionante almeno ogni 3 mesi) così il cliente lo prova e dà feedback reali.

2. Che cos'è una User Story (US)?

- È una breve descrizione di come il sistema si comporterà, ma vista con gli occhi dell'utente finale.
- È scritta in linguaggio naturale (niente "tecniche" da programmatori).
- Deve sempre portare un "valore" al cliente, ovvero essergli utile in qualche modo.
- Vengono scritte fisicamente su dei cartoncini o schede formato A6 (10x15 cm).
- Le scrive il cliente (o il Product Owner), al massimo con l'aiuto degli sviluppatori.
- Non pretendono di dire *tutto* nei minimi dettagli: servono proprio come pretesto per far incontrare sviluppatori e cliente e parlarne a voce.
- L'insieme di tutte le storie da fare si chiama "Backlog".
- **Attenzione:** le User Stories NON sono la stessa cosa dei "Casi d'Uso". I casi d'uso sono documenti più lunghi, esaustivi e scritti dagli analisti, mentre le US sono semplici e in mano al cliente.

3. La regola d'oro per una buona US: INVEST

Per capire se una User Story è scritta bene, deve rispettare le caratteristiche dell'acronimo **INVEST**:

- **Indipendente**: deve poter essere realizzata senza dipendere troppo dalle altre storie.
- **Negoziabile**: non è un contratto rigido, i dettagli si possono e si devono concordare a voce col cliente.
- **Valuable** (Di Valore): deve dare un beneficio o un'informazione utile a chi la usa.
- **Estimable** (Stimabile): gli sviluppatori devono poter stimare quanto tempo ci vorrà a programmarla.
- **Size** (Della giusta dimensione): non deve essere un lavoro immenso.
- **Testable** (Testabile): deve essere possibile scrivere un test per capire se è stata fatta bene.

4. Il formato classico di una scheda

- **Il testo della storia** segue quasi sempre questo schema: *"In qualità di [ruolo], voglio [azione] in modo da [motivazione]"*. Esempio: "In qualità di cliente, voglio cancellare una prenotazione in modo da ottenere il rimborso".
- **Priorità**: è il cliente a decidere cosa è più urgente (di solito con una scala da 1 a 3).
- **Story Points**: è la stima dello sforzo necessario, calcolata dai programmatori.
- **Test di Accettazione (TA)**: servono a verificare che la storia funzioni. Spesso seguono lo schema *"Dato che [contesto], quando [azione], allora [risultato]"*. Sono test scritti dal cliente, ma che i programmatori cercano di automatizzare (ad esempio usando software come Selenium). Una storia si considera finita *solo* se passa tutti i TA.

5. Stime, Epiche e "Spike"

- Il tempo per sviluppare una storia si misura in "Punti Storia" (spesso 1 punto equivale a un giorno o mezza giornata di lavoro, ma dipende dal team).
- **Epiche**: se una storia è troppo grande per essere stimata o se richiede più di due settimane di lavoro (es. più di 10 punti), viene chiamata "Epica". Va spezzata in tante US più piccole, indipendenti e fattibili in pochi giorni.

- **Spike solution:** cosa succede se i programmatori non sanno stimare una storia perché non conoscono bene la tecnologia? Fanno un esperimento. Creano un prototipo veloce per chiarirsi le idee sui tempi (gestendo così il rischio) e poi di solito buttano via il codice scritto.

6. Il ciclo di vita nel processo Agile

- All'inizio i requisiti si raccolgono tramite riunioni. Si può fare in modo strutturato partendo dagli obiettivi ("Goal-oriented") oppure a ruota libera ("Scattergun").
- Le US vanno nel "Product Backlog" globale.
- All'inizio di una singola iterazione (chiamata "Sprint", che dura 1-2 settimane), si scelgono le US da fare in quel lasso di tempo (e finiscono nello "Sprint Backlog").
- Il cliente le spiega bene a voce, i programmatori lavorano, e alla fine dello Sprint il cliente esegue i test per validare il lavoro.
- *Nota sui requisiti non funzionali (es. "il sito deve caricare in 300 ms"):* anche se non sono perfetti per questo formato, possono essere infilati all'interno delle User Stories come vincoli aggiuntivi.

Fammi sapere se questo approccio ti è utile e, appena sei pronto, incolla pure il prossimo file!

Eccoti il riassunto del secondo file sul **Testing**. Anche qui ho tolto un po' di ragnatele accademiche per renderlo il più diretto e digeribile possibile!

1. Cos'è il Software Testing e perché ci salva la vita

- Il testing è una procedura in cui facciamo girare il sistema (chiamato SUT, System Under Test) per scovare i difetti.
 - In pratica, si confronta il "Risultato atteso" con il "Risultato ottenuto".
 - Trovare un problema in produzione (quando il software è già rilasciato) costa dalle 40 alle 100 volte in più rispetto a scovarlo durante lo sviluppo.
 - È letteralmente impossibile fare un "testing esaustivo", cioè provare tutte le infinite combinazioni di input possibili.
 - Bisogna quindi testare "il prima possibile e il più spesso possibile", creando test automatizzati e ripetibili fin da quando si hanno i requisiti.
 - Il vocabolario degli intoppi prevede quattro termini specifici.
 - **Mistake**: l'errore umano che causa il problema.
 - **Fault (o Difetto)**: il pezzo di codice o la definizione sbagliata scritti nel programma.
 - **Failure (Fallimento)**: il sistema si blocca o non fa quello che deve fare.
 - **Error**: la differenza matematica o logica tra il valore calcolato e quello che dovrebbe essere corretto.
-

2. Verifica vs Validazione (Le due "V")

- La **Verifica** risponde alla domanda: "Stiamo costruendo il prodotto nel modo giusto?". Controlla se il software rispetta le specifiche tecniche e di progetto stabilite all'inizio.
 - La **Validazione** risponde alla domanda: "Stiamo costruendo il prodotto giusto?". Controlla se il software risolve davvero i bisogni e le aspettative del cliente.
 - La Validazione è molto più difficile della Verifica, perché le specifiche iniziali potrebbero essere perfette sulla carta, ma inutili per le reali esigenze dell'utente.
-

3. Test Case e Test Suite

- Un **Test Case** è una sequenza precisa di passi per verificare se una singola funzione va bene.
 - Deve sempre contenere una serie di input, le condizioni di partenza e il risultato che ci si aspetta.
 - Una **Test Suite** è semplicemente una collezione o un pacchetto che raggruppa tanti Test Case.
-

4. White Box vs Black Box: Le due filosofie

- **White Box Testing (Scatola Trasparente):** Il tester guarda dentro il codice e ne conosce la struttura.
 - Serve a verificare i percorsi logici, i cicli (loop) e le strutture dati interne.
 - Si basa sul concetto di "copertura" (coverage), ovvero la percentuale di codice che viene effettivamente eseguita durante il test.
 - Spesso è fatto dallo sviluppatore stesso (es. Unit Test) e cattura fino al 65% dei bug.
 - Il difetto del White Box è che non può trovare funzionalità mancanti, perché se un requisito non è stato programmato, non c'è codice da testare.
 - **Black Box Testing (Scatola Nera):** Il tester non ha idea di come sia scritto il codice, guarda solo gli input che inserisce e gli output che escono.
 - Si basa esclusivamente sui requisiti ed è perfetto per simulare il comportamento dell'utente finale.
 - Può scovare funzioni mancanti, problemi di interfaccia e difetti di prestazioni.
 - I due approcci non sono in competizione, ma sono complementari e vanno usati entrambi.
-

5. I Livelli di Testing (Dal piccolo al grande)

- **Unit Testing (White Box):** Si testa un singolo pezzettino di codice (funzione o metodo) isolandolo dal resto, spesso usando oggetti finti (mock) per simulare l'ambiente.

- **Integration Testing:** Si uniscono i moduli per vedere se "parlano" bene tra loro. Spesso gli errori si nascondono qui, magari per parametri passati nell'ordine sbagliato.
 - **Functional Testing (Black Box):** Assicura che le funzionalità descritte nei requisiti funzionino.
 - **System Testing (Black Box):** Si testa tutto il sistema completo per vedere se rispetta requisiti, prestazioni, sicurezza e usabilità.
 - **Stress Testing:** Si "tira il collo" al sistema caricandolo di lavoro fino al punto di rottura per vederne la stabilità.
 - **Acceptance Testing:** È il test finale di validazione in cui il cliente verifica se il sistema rispetta i criteri per essere accettato e pagato.
 - **Regression Testing:** Si rifanno i test dopo aver modificato il codice per assicurarsi di non aver rotto nient'altro nel frattempo (effetti collaterali).
 - **Beta Testing:** Si dà il software quasi pronto agli utenti reali per farglielo usare e raccogliere le loro segnalazioni di bug.
-

6. Pianificare i test (Test Plan e Three Step Approach)

- Il **Piano di Test** è il documento che descrive come, quando e con quali risorse si faranno i test, oltre ai rischi previsti.
- Un test case si compila con quattro campi chiave: ID, Descrizione (con i passi), Risultato atteso, Risultato effettivo ottenuto.
- L'approccio "**Three Step**" per creare i test parte dai casi d'uso: per ogni caso d'uso si creano degli scenari, per ogni scenario si trovano le condizioni per eseguirlo (quasi-test case), e per ognuno di questi si scelgono i valori esatti di input per creare i Test Case veri e propri.
- **Integrazione Continua (Continuous Integration):** Il codice prodotto dai vari sviluppatori dovrebbe essere unito almeno una volta al giorno, facendo girare subito tutti i test automatici per avere feedback immediati.

Eccoci qua con il riassunto del terzo file! Questo documento si concentra sull'**Agile Testing**, in particolare sulle pratiche nate con l'eXtreme Programming (XP) ma che ormai sono usate da praticamente tutti i team agili.

Come sempre, niente burocratese, andiamo dritti al sodo.

1. La regola d'oro: Automatizzare tutto

- Nell'approccio Agile, i test non si fanno a mano: devono essere automatizzati e scritti proprio sotto forma di codice.
- I test devono essere facili da eseguire per chiunque e in qualunque momento.
- Questo dà un coraggio enorme agli sviluppatori: se aggiungi o modifichi una funzione e i test automatici continuano a passare, sai di non aver sfasciato il sistema (questo si chiama "test di regressione").
- **Pro e Contro:** Fanno risparmiare tempo e soldi perché evitano infinite sessioni di debug "alla cieca", rendendo il sistema molto più affidabile. Il contro? Scrivere questi test costa tempo e fatica, quindi serve che il manager conceda il tempo per farli, vedendoli come un investimento a lungo termine.

2. Il Manifesto dei Test Automatici (Come scriverli bene)

Secondo Meszaros, i test automatici devono seguire queste linee guida:

- **Concisi e Chiari:** Semplici da capire, tanto da funzionare come documentazione utile a spiegare come si comporta il software.
- **Self-Checking:** Il test deve dire in automatico se è passato o fallito, senza che un essere umano debba stare lì a interpretare i risultati.
- **Ripetibili e Robusti:** Devi poterli lanciare all'infinito ottenendo sempre lo stesso risultato. Non devono fallire a causa di internet assente, dell'ordine in cui vengono eseguiti o se il test precedente è andato male.
- **Sufficienti e Necessari:** Devono coprire tutti i requisiti del software, ma senza testare cose superflue o fuori bersaglio.
- **Efficienti e Specifici:** Devono essere veloci da eseguire e, in caso di fallimento, devono farti capire immediatamente quale funzionalità si è rotta.

3. TDD (Test-Driven Development): Prima il test, poi il codice

Questa è una pratica potentissima: **si scrivono i test prima ancora di scrivere il programma!**

- Il TDD non serve solo a testare, ma è un vero strumento per progettare il codice. Se fai fatica a scrivere un test, significa che c'è un problema di progetto, non di programmazione.
- Tutto il codice deve avere dei test unitari e questi devono passare al 100%. Se si scopre un bug, la prima cosa da fare è scrivere un test che lo scovi, e solo dopo correggere il codice.

- **I 6 passi del TDD:**

1. Fai un mini-progetto capendo cosa ti serve.
2. Scrivi il test automatico per quella funzione.
3. Fai girare il test e verifichi che fallisca (ovvio, il codice non c'è ancora!).
4. Scrivi il codice minimo indispensabile per far passare il test.
5. Fai girare di nuovo il test: ora deve passare.
6. Fai *refactoring*: pulisci e migliora il codice appena scritto.

- **Perché farlo?** Previene la programmazione selvaggia ("cowboy coding"), spinge a scrivere codice ben strutturato, crea fiducia tra i programmatori e impone un ritmo di lavoro sano ed efficiente.

4. Test Unitari vs Test di Accettazione

Il file fa una bella distinzione tra queste due bestie:

- **Test Unitari:**

- Testano il singolo modulo di codice (verificano il "come" funziona).
- Li scrivono gli sviluppatori per comunicare con altri sviluppatori.
- Devono passare sempre al 100% e favoriscono le modifiche sicure al codice (refactoring).

- **Test di Accettazione:**

- Sono test "Black Box" (si guarda l'effetto esterno) e verificano se il sistema risponde ai requisiti (il "cosa" deve fare).
- Li scrivono il cliente o il Product Owner (PO) insieme ai programmatori, partendo direttamente dalle User Stories.
- Seguono il formato: ID, Descrizione, Risultato Atteso, Risultato Ottenuto.
- Quando un test di accettazione viene superato, il cliente considera quella specifica User Story completata.

5. Agile vs Tradizionale in pillole

- **Metodo Tradizionale (Waterfall):** I test li fa un team a parte alla fine di tutto lo sviluppo, spesso sono manuali e la loro funzione è solo di verifica finale.
- **Metodo Agile:** I test si scrivono prima o durante lo sviluppo (insieme alle User Stories) collaborando col cliente. Vengono eseguiti in ogni "Sprint" (spesso tutti i giorni in automatico) e servono sia per progettare che per verificare. Questo

rispecchia i principi Agili: collaborazione, software funzionante e risposta rapida ai cambiamenti.

Eccoti il riassunto del quarto file! Questo entra nel vivo della pratica, spiegandoci *come* si automatizzano i test, con un occhio di riguardo al mondo Python. Come sempre, te lo scompongo in modo chiaro e diretto.

1. BDD (Behavior-Driven Development) e il linguaggio Gherkin

- Il BDD è un approccio in cui si descrive cosa deve fare il sistema partendo dai risultati visibili e dal valore per l'utente.
- Invece di parlare di codice, ci si fa una domanda pratica: "Dato questo contesto, cosa succede se l'utente fa questa azione?".
- Per scrivere queste regole si usa il linguaggio **Gherkin** (che letteralmente significa "cetriolo aromatizzato", perché deriva dal framework storico chiamato "Cucumber").
- Gherkin permette di descrivere i Test di Accettazione in modo leggibile da tutti (usando l'inglese o l'italiano), definendo le regole di business.
- **Feature:** È il termine che descrive la funzionalità generale o l'area di business (es. "Gestione login").
- **Scenario:** Descrive il caso specifico da testare.
- **Given (Dato):** Imposta la situazione di partenza o preconditione.
- **When (Quando):** È l'azione compiuta dall'utente.
- **Then (Allora):** È il risultato visibile che ci aspettiamo di ottenere.
- **Background:** Contiene le precondizioni che valgono per *tutti* gli scenari di quella specifica Feature, ed evita di dover ripetere sempre gli stessi passi.
- **Scenario Outline ed Examples:** Permettono di eseguire lo stesso test più volte, passandogli una tabella di dati diversi ad ogni ciclo (es. provare il login con più email).
- **Tag:** Etichette (come @smoke, @api, @ui) che servono a classificare ed eseguire solo un certo gruppo mirato di test.
- *Nota per l'esame:* Il BDD è diverso dai normali Casi d'Uso. I Casi d'Uso descrivono attori e flussi completi, mentre il BDD si concentra su esempi pratici e comportamenti verificabili (spesso da un singolo Caso d'Uso nascono molti scenari BDD).

2. La famiglia xUnit e l'anatomia di un Test

- **xUnit** è la famiglia "storica" di tutti i framework per i test unitari (inventato originariamente per Smalltalk e reso famoso con JUnit in Java).

- Si basa su quattro concetti base (pattern):
- **Fixture:** È l'ambiente in cui gira il test. Inizializza i dati e le variabili prima di partire (col metodo setUp) e ripulisce le risorse alla fine (col metodo tearDown).
- **Test-Case:** È il singolo esperimento atomico e ripetibile.
- **Check:** Sono le funzioni di controllo (le famose "asserzioni", tipo controllare che $2+2=4$ che determinano se un test è passato.
- **Test Suite:** Un raggruppamento organizzato di più Test Case lanciati in blocco.
- **Attenzione ai termini Error vs Failure:** In xUnit, "Error" significa che il programma è crashato generando un'eccezione, mentre "Failure" significa che il programma non si è bloccato, ma il risultato del controllo sui dati è sbagliato. Questa definizione è diversa da quelle standard dell'IEEE viste nel file precedente!.

3. I Mock Object

- Sono oggetti "finti" usati nei test per simulare parti del sistema che non sono ancora state implementate o che sono scomode da usare (come un database reale o un'API esterna).
- Servono a isolare perfettamente il frammento di codice che stai testando, rendendo l'esecuzione molto più rapida e prevedibile.

4. Il panorama dei Framework in Python

Il file fa una carrellata degli strumenti più usati oggi in Python per fare testing:

Per i Test Unitari:

- **unittest:** È il modulo base già integrato nella libreria standard di Python. È molto solido, ma un po' "verboso" e rigido. Ideale per progetti legacy o ambienti aziendali formali.
- **pytest:** Oggi è la scelta predefinita e il più usato. È pulito, leggibile, non obbliga a creare classi per ogni test, gestisce alla perfezione le Fixture condivise e ha tantissimi plugin utili.
- **nose2:** Un'estensione di unittest, utile se hai già un vecchio progetto e vuoi aggiungerci dei plugin, ma per i progetti nuovi si preferisce pytest.

Per i Test di Accettazione:

- **behave:** È la versione Python orientata al BDD. Prende i file scritti in Gherkin (con i Given/When/Then) e li "aggancia" al codice Python sottostante. Perfetto per far comunicare sviluppatori e clienti.
- **Robot Framework:** Usa un approccio basato su "parole chiave" per creare test end-to-end (E2E) estremamente chiari e leggibili anche da non-programmatori.

Per i Test End-to-End e Interfacce Web:

- **Selenium:** È il framework storico e mastodontico per simulare un utente che apre e clicca su un browser web.
- **Playwright:** È l'alternativa moderna. È più veloce, gestisce meglio i browser di ultima generazione e la combo con pytest è la scelta top per i nuovi progetti web in Python.

Se hai altri file o vuoi chiarimenti su questi framework, io sono qua!

Eccoci al quinto file! Questo è super moderno e pratico: parla di come usare l'Intelligenza Artificiale (i famosi LLM, come ChatGPT) per farsi dare una mano a scrivere gli **Unit Test**.

Come sempre, niente giri di parole. Ecco il succo del discorso:

1. Perché usare un LLM per i test (e come inquadrarlo)

- **Risparmio di tempo:** Ti toglie la rottura di dover scrivere la primissima bozza del test da zero.
- **Ti salva dagli "ops":** È bravissimo a suggerirti casi limite (edge case), percorsi felici (happy path) e casi di errore a cui magari non avevi pensato.
- **Scrive la "burocrazia" del codice:** Aiuta a creare velocemente la struttura di base come fixture, mock object e test parametrizzati.
- **Ottimo per il codice vecchio:** Se ti ritrovi per le mani del codice "legacy" (vecchio e senza test), l'LLM è una manna dal cielo per coprirlo rapidamente.
- **La Regola d'Oro:** L'LLM va trattato come un *pair programmer* (un collega che programma con te), NON come un oracolo magico da accettare alla cieca.

2. Il flusso di lavoro ideale in 5 passi

Non puoi semplicemente incollare il codice e dirgli "fammi i test". Devi guidarlo così:

1. **Dai il contesto:** Incolla il codice, spiega l'obiettivo e digli che framework usare (es. pytest).
2. **Fatti elencare i casi:** Chiedigli prima di farti un elenco di cosa vuole testare (casi positivi, errori, ecc.).
3. **Genera i test:** Fagli scrivere il codice vero e proprio usando mock e fixture se servono.
4. **Esegui e correggi:** Prendi il codice, fallo girare sul tuo computer e correggi gli inevitabili errori o import sbagliati.
5. **Raffina:** Metti a posto i nomi, controlla che sia leggibile e assicurati che copra davvero quello che serve.

3. Come scrivere il Prompt perfetto

Per ottenere test utili, devi dare in pasto all'IA le cose giuste.

- **Il minimo indispensabile:** Passagli il codice, spiegagli il comportamento atteso, digli quale framework vuoi, la versione di Python e metti dei paletti chiari (ad esempio: *"Non usare chiamate di rete reali, usa dei mock"*).

- **Il trucco del mestiere:** I prompt migliori sono quelli iterativi. Invece di chiedere tutto subito, digli: *"Prima fammi un elenco puntato di 6 casi di test. Poi scrivi i test in pytest usando pytest.mark.parametrize"*.

4. I rischi (A cosa stare molto attenti!)

L'IA a volte ha le allucinazioni, quindi occhio a queste tre trappole:

- **Assunzioni sbagliate:** L'LLM potrebbe inventarsi comportamenti che il tuo codice non ha, oppure sbagliare completamente i percorsi per importare i file.
- **Coverage (Copertura) "Finta":** Potrebbe scriverti test che passano perfettamente, ma che in realtà non stanno verificando un bel niente di utile.
- **Test fragili:** Potrebbe creare asserzioni troppo rigide e legate a come è scritto il codice internamente; se cambi una virgola, il test si rompe.

5. In sintesi: Buone pratiche

- Fai sempre girare il test localmente per validarlo: la revisione umana è insostituibile.
- Mantieni i test piccoli, indipendenti e leggibili.
- Inizia con funzioni semplici o puro codice (dove i mock sono facili da usare) per massimizzare il risparmio di tempo.
- **Ricorda:** L'obiettivo è farti scrivere test *utili* in modo *più veloce*, non delegare all'IA la qualità del tuo software.

Passami pure il prossimo file quando sei pronto!

Eccoci qua col sesto file! Questo è un vero e proprio tuffo nel cuore delle **Metodologie Agili** e dell'**eXtreme Programming (XP)**. Preparati, perché qui si smonta il modo "vecchio" di fare software e si capisce davvero come lavorano i team moderni.

Niente giri di parole, andiamo dritti al sodo!

1. Perché è nato l'Agile? (Addio al "Plan-driven")

- Prima si usavano metodi tradizionali (come il modello a "Cascata") basati su tanta pianificazione, contratti rigidi e tonnellate di documentazione.
- Il problema? L'economia di Internet richiede velocità e flessibilità. I clienti erano stupefatti dei ritardi continui e della bassa qualità del software, anche perché all'inizio di un progetto è quasi impossibile sapere *esattamente* cosa si vuole.
- Il software non è un prodotto industriale da "catena di montaggio": è malleabile, logico, immateriale e il costo sta tutto nel pensarlo/progettarlo, non nel "fabbricarlo".
- Così nascono le Metodologie Agili (AM): processi flessibili, adattivi e incrementali che non sono affatto "programmazione selvaggia", ma richiedono molta disciplina per mantenere alta la qualità.

2. Il Manifesto Agile (2001)

I pionieri dell'Agile si sono riuniti e hanno stabilito quattro valori fondamentali (le cose a sinistra contano più di quelle a destra):

- **Gli individui e le interazioni** *più che* i processi e gli strumenti.
- **Il software funzionante** *più che* la documentazione infinita.
- **La collaborazione col cliente** *più che* la negoziazione dei contratti.
- **Rispondere al cambiamento** *più che* seguire un piano rigido.
- *Principi chiave:* rilasciare software funzionante spesso, accogliere i cambiamenti anche in ritardo, far lavorare insieme sviluppatori e clienti quotidianamente, auto-organizzazione del team e puntare alla semplicità.

3. L'eXtreme Programming (XP) e i suoi Valori

L'XP è stata inventata nel 1997 da Kent Beck (che la usò per salvare un disastroso progetto di stipendi per la Chrysler, il progetto C3, risolvendolo in soli 2 anni con pochi programmatori). È una metodologia che prende tecniche di sviluppo buone e le spinge all'estremo. Si basa su **5 valori**:

1. **Comunicazione:** I problemi nascono quando non ci si parla. La forma migliore è parlarsi faccia a faccia.

2. **Semplicità:** Fai la cosa più semplice che possa funzionare.
3. **Feedback:** Misura sempre come sta andando tramite test continui e contatto col cliente.
4. **Coraggio:** La capacità di stravolgere e migliorare il codice (refactoring) senza paura.
5. **Rispetto (aggiunto dopo):** Senza rispetto per i colleghi e per il progetto, non si va da nessuna parte.

4. Le 12 Pratiche dell'XP (Il cuore del metodo)

Queste sono le regole pratiche che (se applicate insieme) fanno funzionare l'XP:

- **Planning Game:** Si pianifica in modo iterativo usando le User Stories e stimando la velocità del team.
- **Rilasci brevi:** Si rilascia ogni 2-3 mesi, suddivisi in iterazioni (sprint) di poche settimane.
- **Metafora:** Avere una visione/storia comune per spiegare il sistema a tutti.
- **Semplicità di progetto (YAGNI):** "You Aren't Gonna Need It!" (Non ti servirà!). Non scrivere MAI codice per cose che *forse* ti serviranno in futuro. Scrivi solo il necessario per oggi.
- **Testing Continuo:** I test automatici si scrivono *prima* del codice (TDD).
- **Refactoring:** Migliorare continuamente l'architettura del codice senza cambiarne il comportamento esterno.
- **Pair Programming (Programmazione a coppie):** Si programma in due su un solo PC. Uno scrive e pensa al dettaglio tattico, l'altro guarda lo schermo e pensa alla strategia globale e ai test. Le coppie ruotano continuamente.
- **Integrazione Continua:** Il codice nuovo si unisce a quello vecchio e si testa almeno una volta al giorno.
- **Proprietà collettiva del codice:** Chiunque può (e deve) mettere le mani su qualsiasi parte del sistema.
- **Cliente sul posto:** Il cliente (o chi fa le sue veci) lavora con il team ogni giorno.
- **Ritmo sostenibile:** Niente straordinari logoranti (si lavora 40 ore e stop), altrimenti si perde lucidità e si fanno danni.
- **Standard di codifica:** Tutti scrivono il codice usando lo stesso stile, in modo che sia "terso" e così chiaro da fare esso stesso da documentazione.

5. Regole d'oro del Planning Game

- Un progetto ha **4 variabili**: Funzionalità, Tempo, Risorse e Qualità.
- La qualità è sempre fissa (deve essere alta).
- **Legge di Brooks (Risorse non lineari)**: Se un progetto è in ritardo e aggiungi nuove persone al team, *il progetto andrà ancora più in ritardo* (perché devi fermarti a formare i nuovi, c'è più caos comunicativo, ecc.).
- Il cliente fissa le Funzionalità e il Tempo, gli sviluppatori dicono quante cose riescono a fare (Velocità). Il cliente non può comandare su tutte e 4 le variabili.

6. Punti Storia e Burndown Chart

- **Story Points**: Per capire quanto tempo ci vorrà a fare una User Story, si stimano i "Punti Storia". Sono unità relative (è più facile dire "questo lavoro è grosso il doppio di quell'altro" anziché dire "ci vorranno esattamente 13 ore").
- **Burndown Chart**: È un grafico fondamentale che mostra le ore (o i punti) *rimanenti* da fare. Ogni giorno il grafico scende. Se la linea va a zero prima del previsto, siete in anticipo; se sta sopra, siete in ritardo. Aiuta a non poter mentire sullo stato dei lavori!.

7. Strumenti utili e Ruoli

- **Schede CRC (Classe, Responsabilità, Collaborazione)**: Strumento usato per passare dai requisiti (storie) a come strutturare gli oggetti del sistema.
- **Spike Solution**: Un programmino veloce "usa e getta" fatto solo per capire come risolvere un problema tecnico e ridurre i rischi.
- **Ruoli XP**: Ci sono il Programmatore, il Cliente, il Tester, il Tracker (che monitora le metriche), il Coach (che fa rispettare l'XP) e il Manager.

Ecco il riassunto dell'ultimo file su **DevOps**, strutturato in modo chiaro e facile da memorizzare per l'esame.

1. Cos'è DevOps? (Oltre il muro tra Dev e Ops)

- **Il problema:** Storicamente, chi scriveva il software (i *Developers* o Dev) voleva rilasciare continuamente nuove funzionalità, mentre chi gestiva i server e l'infrastruttura (le *Operations* o Ops) voleva stabilità e meno cambiamenti possibili. Questo creava un "muro", lentezze e conflitti ("Sul mio PC funziona, ora è un problema vostro!").
 - **La soluzione:** DevOps (termine coniato nel 2009) è la metodologia che fonde e automatizza lo sviluppo e le operazioni IT.
 - **L'obiettivo:** Non solo rilasciare software più spesso, ma accorciare drasticamente il tempo che passa tra quando scrivi una riga di codice e quando questa gira in modo affidabile in produzione per l'utente finale. È, in pratica, l'applicazione dei metodi Agili all'amministrazione dei sistemi.
-

2. I protagonisti: Dev, QA e Ops

DevOps non cancella i ruoli precedenti, ma li coordina:

- **Dev (Sviluppo):** Programmatori e architetti. Vogliono massimizzare le nuove funzionalità senza introdurre difetti. Usano Git, VS Code, Docker, ecc.
 - **QA (Quality Assurance):** Tester. Il loro scopo è validare le funzionalità e prevenire che vecchi bug tornino (regressioni), aumentando la fiducia nei rilasci. Usano Selenium, pytest, Postman, ecc.
 - **Ops (Operations):** Sistemisti e Cloud Engineer (spesso oggi chiamati SRE, Site Reliability Engineer o DevOps Engineer). Vogliono stabilità, sicurezza e rilasci continui. Gestiscono server, backup, sicurezza, scaling (adattarsi al carico di utenti) e ripristini. Usano Kubernetes, Terraform, piattaforme Cloud.
 - Oggi le Ops non sono più burocratiche e lente: usano strumenti moderni e gestiscono l'infrastruttura scrivendola sotto forma di codice ("Infrastructure as Code").
-

3. I principi CALMS

Per capire se stai facendo DevOps, ricordati l'acronimo CALMS:

- **C (Culture):** Cultura basata sulla collaborazione e non sui silos isolati.

- **A (Automation):** Automatizzare tutto il possibile per eliminare il lavoro manuale ripetitivo (CI/CD).

- **L (Lean):** Usare i principi "snelli" per velocizzare e standardizzare i processi.
 - **M (Metrics):** Misurare tutto per potersi migliorare sempre.
 - **S (Sharing):** Condividere i successi e i fallimenti con tutti per crescere insieme.
-

4. Il Ciclo di Vita (Loop Infinito)

Il ciclo DevOps non finisce mai (spesso è disegnato come un simbolo dell'infinito). Le fasi principali sono:

1. **Plan:** Pianificare basandosi sul business e sui feedback del ciclo precedente.
 2. **Code & Build:** Scrivere il codice, usare il versionamento (Git) e fare la "build" (creare l'eseguibile o il container). Deve essere veloce e affidabile.
 3. **Test:** Test automatici continui (di accettazione, regressione, sicurezza).
 4. **Package & Release:** Preparare la versione per la distribuzione (in un ambiente di preproduzione), gestire dipendenze e orchestrazione. Rilasciando poco e spesso, il rischio di rompere tutto crolla.
 5. **Configure:** Configurare l'ambiente di destinazione usando "Infrastructure as Code" (così si elimina il problema del "sul mio computer funzionava").
 6. **Monitor:** Osservare il sistema in produzione (prestazioni e risposte degli utenti) per capire se ci sono problemi e passare i dati alla fase di "Plan" per il prossimo ciclo.
-

5. Metriche: Come si misura il successo?

DevOps usa metriche precise:

- **Lead time:** Il tempo totale da quando un cliente chiede una cosa a quando è live in produzione.
 - **Cycle time:** Il tempo in cui si sta effettivamente lavorando a quella cosa.
 - **Throughput:** Quante cose si riescono a completare in un tot di tempo.
 - **Change failure rate:** La percentuale di aggiornamenti che causano disastri e vanno annullati (rollback).
 - **MTTR (Mean Time To Recovery):** Quanto tempo ci mette a far ripartire il sistema quando qualcosa si rompe.
-

6. Esempio Pratico (La Pipeline CI/CD)

Se usi DevOps, il flusso ideale è:

1. Il programmatore scrive il codice e fa "push" su GitHub.
2. Questo scatena in automatico la **CI (Continuous Integration)**: un server installa le librerie, lancia i test e crea il pacchetto software.
3. Se tutto va bene, parte la **CD (Continuous Deployment)**: il pacchetto viene pubblicato automaticamente sul server reale senza interventi umani.
4. Se c'è un errore (un test fallisce o il server si blocca), il sistema avvisa subito il team (es. via Slack).

Spero che questa sintesi di tutti i documenti ti sia di grande aiuto per il parziale! In bocca al lupo per lo studio, e se ti serve un ripasso finale o hai dubbi su qualche concetto incrociato, chiedi pure!

Eccoci al gran finale (almeno per questo giro di slide)! Questo ottavo file ci porta nel mondo del **Lean Software Development (LSD)** e del metodo **Kanban**. È un approccio che punta a fare "di più con meno", eliminando tutto ciò che non serve.

Ecco il riassunto sgrassato e dritto al punto:

1. Le radici: Cos'è il Lean Software Development?

- Nasce dalle pratiche industriali della Toyota degli anni '80 (Lean Manufacturing), riadattate per il software da Mary e Tom Poppendieck nei primi anni 2000.
- Non ti dice *come* programmare (come fa l'XP), ma si concentra su *come e quando* far scorrere il lavoro.
- L'obiettivo primario del Lean è aumentare il valore per il cliente tagliando senza pietà gli sprechi.

2. I 7 Principi d'Oro del Lean

- **Elimina gli sprechi (Waste/Muda):** Lo spreco è tutto ciò che non aggiunge valore per il cliente. Nel software gli sprechi sono il lavoro lasciato a metà, la burocrazia inutile, i difetti, le attese e, soprattutto, le funzionalità extra. Uno studio ha dimostrato che il 50% delle funzionalità sviluppate in un sistema non viene *mai* usato.
- **Amplifica l'apprendimento:** Si sviluppa a piccoli lotti per testare le idee, raccogliere feedback e imparare dagli errori il prima possibile.
- **Decidi il più tardi possibile:** Mantenere le opzioni aperte finché non si hanno dati certi. Nel software agile è facile, perché i requisiti non ancora implementati si possono cambiare a costo zero.
- **Consegna il più velocemente possibile:** Meno tempo passa dall'idea al rilascio, meglio è. Si fa lavorando con il sistema "pull" e gestendo bene le code di lavoro.
- **Dai potere al team:** Addio ai capi che micro-gestiscono. Le persone che fanno il lavoro pratico sono quelle che devono avere il potere di prendere le decisioni.
- **Costruisci l'integrità all'interno:** Il sistema deve essere coerente e solido fin dalla base, usando test continui e refactoring.
- **Considera il tutto:** Bisogna ottimizzare l'intero sistema, non solo il proprio piccolo pezzettino, curando le cause dei problemi e non i sintomi.

3. Kanban: Il tabellone magico

- "Kanban" in giapponese significa "segnale visuale" ed è lo strumento principe per gestire il flusso di lavoro.
- Utilizza un tabellone fisico o virtuale diviso in colonne che rappresentano le fasi del lavoro (es. Backlog, Selected, Develop, Deploy, Live).
- Le funzionalità o i "task" sono scritti su dei cartellini (Card) che si spostano da sinistra verso destra.
- Funziona con un approccio **Pull** (Tirare): non si "spinge" nuovo lavoro addosso agli sviluppatori, ma sono loro che "tirano" un nuovo cartellino dalla fase precedente solo quando hanno finito quello a cui stavano lavorando e hanno spazio libero.

4. La regola fondamentale del Kanban: Limitare il WIP

- **WIP (Work In Progress):** È il numero di attività iniziate ma non ancora finite.
- Il Kanban impone un limite matematico al WIP per ogni colonna. Ad esempio, non si possono avere più di 4 feature in fase di "Sviluppo" contemporaneamente.
- Fare troppe cose insieme (multitasking) è un'illusione che genera ritardi, stress e difetti (il cervello impiega circa 23 minuti per ritrovare la concentrazione dopo un'interruzione).
- Limitare il WIP fa emergere subito i colli di bottiglia e accelera le consegne.

5. Il vocabolario delle Metriche

- **Lead Time:** Il tempo totale da quando un'idea entra nel backlog a quando viene completata.
- **Cycle Time:** Il tempo da quando si *inizia* effettivamente a lavorare su una specifica feature a quando la si finisce.
- **Throughput:** La produttività del sistema, ovvero quanti elementi vengono finiti in una determinata unità di tempo.
- **Legge di Little:** $Cycle\ Time = WIP / Throughput$. Dimostra matematicamente che, a parità di produttività, se si riducono le cose fatte in contemporanea (WIP), si riduce automaticamente il tempo di consegna (Cycle Time).
- **Cumulative Flow Diagram (CFD):** È un grafico a fasce colorate utilissimo. Se lo spessore verticale di una fascia diventa enorme, significa che c'è troppo lavoro

accumulato in quella fase (collo di bottiglia). Se una linea è piatta, significa che nessuno sta producendo nulla.

6. Kanban vs Scrum (e l'ibrido Scrumban)

- **Le iterazioni:** Lo Scrum lavora a blocchi fissi (Sprint di 1-4 settimane), mentre il Kanban è un flusso continuo senza interruzioni.
- **I ruoli:** Lo Scrum impone ruoli rigidi (Product Owner, Scrum Master), il Kanban non prescrive nessun ruolo specifico.
- **I cambiamenti:** Nello Scrum non si può cambiare nulla una volta iniziato lo Sprint, nel Kanban si può aggiungere qualcosa in qualsiasi momento, purché non si superi il limite del WIP.
- **Scrumban:** È una fusione dei due mondi. Prende la struttura a Sprint dello Scrum e ci aggiunge il tabellone visivo e i limiti rigorosi sul WIP del Kanban.

Eccoci all'ultimo (per ora) tassello del nostro puzzle! Questo file, **Scrum e Tendenze Agili**, è diviso in due parti: la prima spiega il famosissimo framework Scrum, e la seconda ci dà uno sguardo sul futuro, parlando di come l'Agile si sta evolvendo oggi, persino con l'Intelligenza Artificiale.

Andiamo a spaccettarlo!

1. La mentalità Scrum e l'OODA Loop

- Scrum è nato nel 1990 ed è oggi la metodologia Agile più popolare in assoluto. A differenza dell'XP (che ti dice *come* programmare), Scrum si concentra su *come gestire il progetto*.
- Si basa sull'empirismo: si prende una decisione in base a ciò che si impara dall'esperienza, usando cicli di feedback continui.
- Curiosità: il modo di pensare di Scrum deriva dall'**OODA Loop** (Osserva, Orientati, Decidi, Agisci), una strategia militare inventata dal pilota John Boyd. Il concetto è: vince chi riesce a cambiare e adattarsi alla situazione più velocemente del proprio avversario.

2. Polli, Maiali e i Ruoli di Scrum

In Scrum c'è una storiella famosa per dividere i ruoli: un pollo e un maiale vogliono aprire un ristorante chiamato "Uova e Pancetta". Il maiale rifiuta dicendo: "Io sarei completamente coinvolto, tu saresti solo interessato!". In Scrum funziona così:

- **I Maiali (Il Team Scrum, totalmente coinvolti):**
 - **Product Owner (PO):** Decide *cosa* fare. Gestisce i requisiti, fissa le priorità, e accetta o rifiuta il lavoro finito.
 - **Scrum Master (SM):** È il facilitatore. Non è il capo che assegna i compiti, ma colui che si assicura che le regole Scrum vengano rispettate e che rimuove gli ostacoli (blocchi) dal percorso del team.
 - **Team di Sviluppo:** Un gruppo auto-organizzato e "cross-funzionale" (ha tutte le competenze necessarie al suo interno) di 5-9 persone. Loro decidono *come* fare il lavoro.
- **I Polli (Gli Stakeholder, solo interessati):** Clienti o manager che finanziano il progetto, ma che possono interagire solo col PO e lo SM.

3. Il Ciclo di Vita (Gli Eventi)

Scrum vive di cicli prefissati (time-boxed) chiamati **Sprint**, che durano da 1 a 4 settimane. Durante lo Sprint non si possono cambiare i requisiti; al massimo lo si annulla se c'è un disastro. Ecco gli eventi chiave:

1. **Sprint Planning:** Riunione iniziale in cui il PO dice cosa è più importante fare, e il team stima quanto riesce a prendere in carico, creando lo *Sprint Goal* e lo *Sprint Backlog*.
2. **Daily Scrum (Stand-up):** Riunione in piedi di 15 minuti ogni giorno. Ognuno risponde a sole 3 domande: Cosa ho fatto ieri? Cosa farò oggi? Ho degli ostacoli?.
3. **Sprint Review:** A fine Sprint, il team fa una demo del software funzionante al PO e agli stakeholder per farsi approvare il lavoro.
4. **Sprint Retrospective:** Riunione interna per capire cosa ha funzionato, cosa ha fatto schifo e come migliorare il processo nello Sprint successivo.

4. Gli Artefatti (Cosa si produce)

- **Product Backlog:** Il "librone" dei sogni. Contiene tutte le feature, i bug e i requisiti tecnici ordinati per priorità dal PO.
- **Sprint Backlog:** La lista dei compiti (task) specifici scelti per lo Sprint in corso, gestita in autonomia dal Team.
- **Incremento:** Il pezzo di software funzionante e rilasciabile prodotto a fine Sprint.
- **Sprint Burndown Chart:** Il grafico che traccia le ore di sforzo rimanenti giorno per giorno. Se la linea va a zero a fine Sprint, missione compiuta!.

5. Tendenze Agili (Stato dell'Agile 2024/2025) e IA

Le pratiche Agili dominano il mercato (Scrum è primo, seguito da Kanban e XP), ma il report 2024/2025 ci dice cose molto interessanti:

- L'Agile non si usa più solo per il software, ma anche nell'industria e nella sanità.
- **Agile + DevOps** sono ormai un'accoppiata fissa per accelerare le consegne e migliorare la qualità tramite l'automazione.
- **Intelligenza Artificiale:** L'IA sta spopolando. Il 65% usa gli LLM (come ChatGPT) manualmente e il 57% usa assistenti per scrivere codice. Il motivo principale? Efficienza e produttività (77%).

- **I rischi:** Le barriere all'ingresso dell'IA sono principalmente la paura per la sicurezza/privacy (61%) e la mancanza di competenze (46%). Si capisce anche che l'IA impara dai dati: se i dati fanno schifo, l'IA farà danni su larga scala.
- **Agentic AI (IA Agentica):** È la nuova frontiera. Non più bot passivi a cui fare domande, ma agenti autonomi in grado di prendere decisioni, pianificare e risolvere problemi da soli. I team Agili non ne hanno paura, ma sono desiderosi di guidare questa transizione con pragmatismo e responsabilità.

Eccoci all'ultimo round! Questo file tocca un tasto dolentissimo per ogni programmatore: il **Debito Tecnico** e la sua unica vera cura, il **Refactoring**.

Andiamo a smontare anche questo documento con la solita chiarezza!

1. Il Debito Tecnico (TD): Cos'è e perché fa paura

- Il termine è stato coniato nel 1992 da Ward Cunningham e usa una metafora finanziaria perfetta.
- Nello sviluppo software, fare una scelta "quick & dirty" (veloce e sporca) per sbrigarsi è come chiedere un prestito in banca.
- I bug e la difficoltà di manutenzione sono gli "interessi" che paghi nel tempo.
- Se non restituisci il debito sistemando il codice, la complessità si accumula (interessi composti) e il progetto può fallire perché ogni modifica diventa impossibile o troppo rischiosa.
- **Non tutto il debito è uguale:**
 - Può essere **Prudente** (fatto con consapevolezza e un piano per ripagarlo, come un mutuo) o **Avventato** (fatto alla cieca).
 - Può essere **Deliberato** (scelto apposta per uscire sul mercato in tempo) o **Involontario** (dovuto a inesperienza o disattenzione).

2. Cause, Sintomi e Impatto

- **Perché si crea?** Per colpa della pressione temporale, della mancanza di test (che fa nascere la paura di toccare il codice), dell'assenza di documentazione o di code review, e dei nomi incomprensibili dati alle variabili.
- **Come te ne accorgi (Sintomi)?** Le stime dei tempi saltano sempre, spuntano bug a catena ogni volta che modifichi qualcosa, le build sono lente e ci sono parti di codice che nessuno nel team ha il coraggio di toccare.
- **Impatto sui metodi Agili:**
 - In *Kanban*, il TD allunga le code, aumenta i blocchi e nasconde il vero limite del WIP (Work In Progress).
 - In *Scrum*, rende le stime inaffidabili, erode la vera capacità del team e rende inutili le retrospettive se non si prendono azioni tecniche.

3. La Matrice delle Priorità (Cosa sistemare prima?)

Quando hai accumulato debito, devi decidere come muoverti valutando l'impatto del problema e lo sforzo per sistemarlo:

- **Alto Impatto / Basso Sforzo:** Esegui il refactoring ORA!.
- **Alto Impatto / Alto Sforzo:** Pianificalo nel Backlog come un task ufficiale.
- **Basso Impatto / Basso Sforzo:** Accetta la situazione e monitorala.
- **Basso Impatto / Alto Sforzo:** Ignoralo (almeno per adesso).

4. Il Refactoring: Come pagare il debito

- **Definizione (Martin Fowler):** È il processo di modifica della struttura interna del codice per migliorarla, *senza* alterare il comportamento esterno del programma.
- Non è un'operazione cosmetica, ma l'unico modo per evitare che gli "interessi" del debito distruggano il progetto.
- **Cosa NON è:** Non significa aggiungere funzionalità, non significa risolvere bug e non significa stravolgere l'architettura. Significa solo riorganizzare e semplificare (es. $x^2 - 1$ diventa $(x-1)(x+1)$).
- **Quando farlo?** Non si pianificano settimane intere di solo refactoring. Lo si fa continuamente mentre si aggiungono feature o si risolvono bug.
- **La regola del Tre:** La prima volta scrivi il codice, la seconda volta magari lo duplichi, la terza volta che fai la stessa cosa... ti fermi e fai refactoring!.
- **La regola del Boy Scout:** Lascia sempre il codice un po' meglio di come l'hai trovato.
- **Regola di Pareto:** Cerca di dedicare l'80% del tempo alle nuove feature e il 20% al refactoring.

5. I "Code Smell" (Quando il codice puzza)

- Un *Code Smell* non è per forza un errore catastrofico, ma è un "indicatore" visivo che qualcosa nella struttura non va.
- Esempi classici:
 - **Troppo codice:** Classi enormi, metodi infiniti, codice duplicato.
 - **Poco codice:** Classi "pigre" (lazy) che non fanno quasi nulla.
 - **Nomi incomprensibili (Poor Naming):** Variabili chiamate a, b, x.
 - **Numeri Magici:** Numeri inseriti a caso nel codice (es. `if eta < 18`) invece di usare costanti descrittive (es. `SCONTO_MINORENNE`).

- Risolvere questi smell con il refactoring migliora la leggibilità del 100% e rende la manutenzione molto più semplice. Ricorda la regola d'oro: *"Il codice è letto molto più spesso di quanto viene scritto"*.

6. Processo e Strumenti

- **Il ritmo giusto:** Trovi lo smell $\rightarrow$ decidi come semplificare $\rightarrow$ scrivi il test $\rightarrow$ modifichi a piccoli passi $\rightarrow$ controlli che i test passino ancora. Mai fare il passo più lungo della gamba!.
- Esiste un vero e proprio catalogo di refactoring classici (es. estrarre un metodo, rinominare le variabili, convertire blocchi di if/switch in oggetti polimorfici).
- **Strumenti:** Non fare tutto a mano! Usa le funzioni integrate negli IDE (come Eclipse, PyCharm, VSCode) o plugin specifici (come JDeodorant o Flake8 per Python) che ti suggeriscono e automatizzano le modifiche.